Skill 2 Networks

Day 1: IP Addresses & Subnetting

Files path : davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1

note : Iam using Virtual Environment in terminal and all scripts run on it

1.Write a Python script that validates whether a given IP address is IPv4 or IPv6 > 1.1.py

- IPv4 addresses are validated by checking four numeric parts separated by dots each between 0 and 255

- IPv6 addresses are validated by checking hexadecimal groups separated by colons

- If the input does not match either format, it is considered invalid

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.1.py
Enter IP address: 2001:db8::1
Valid IPv6 address

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.1.py
Enter IP address: 2001:db8:03h4::1
Invalid IP address

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.1.py
Enter IP address: 192.168.1.1
Valid IPv4 address

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.1.py
Enter IP address: 192.255.123.256
Invalid IP address
```

2.Convert an IPv4 address from dotted-decimal to binary format > 1.2.py

- The IPv4 address is first split into four decimal parts

- Each part is converted into an 8-bit binary number

- The binary parts are then joined together using dots to form the final binary representation

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.2.py
Enter IPv4 address: 192.168.1.1
Binary format: 11000000.10101000.00000001.00000001
```

3.Extract all IP addresses from a text file > 1.3.py

- The script reads the text file line by line and splits each line into words

- Each word is checked to see if it matches the IPv4 format

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.3.py
192.168.1.1
10.0.0.5
```

4.Write a function that checks if an IP belongs to a private network > 1.4.py

- The function checks whether an IPv4 address belongs to a private network

- It validates the IP format and then compares it against known private IP ranges such as (10.x.x.x) (172.16–31.x.x) and (192.168.x.x.)

- If the IP matches any of these ranges, it is considered a private IP address

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.4.py
Enter IP address: 192.168.1.4
Private IP address

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.4.py
Enter IP address: 10.0.0.5
Private IP address

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.4.py
Enter IP address: 8.8.8.8
Public IP address

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.4.py
Enter IP address: 172.20.10.3
Private IP address
```

5.Given an IP and subnet mask, calculate the network address and broadcast address > 1.5.py

- The program takes an IP address and a subnet mask as input

- The network address is calculated by applying a bitwise AND operation between the IP address and the subnet mask

- The broadcast address is calculated by applying a bitwise OR operation between the network address and the wildcard mask

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.5.py
Enter IP address: 192.168.1.10
Enter Subnet Mask: 255.255.255.0
Network Address   : 192.168.1.0
Broadcast Address : 192.168.1.255
```

6.Generate a list of all possible IPs in a given subnet (192.168.1.0/24) > 1.6.py

- A /24 subnet means the first three octets are fixed and the last octet represents the host part

- Therefore, generating all IPs in 192.168.1.0/24 is done by looping the last octet from 0 to 255 and building each IP address

- The script generates all possible IP addresses in the 192.168.1.0/24 subnet and prints the first and last five addresses for clarity

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.6.py
First 5 IPs: ['192.168.1.0', '192.168.1.1', '192.168.1.2', '192.168.1.3', '192.168.1.4']
Last 5 IPs: ['192.168.1.251', '192.168.1.252', '192.168.1.253', '192.168.1.254', '192.168.1.255']
```

7.Create a script that checks if an IP is reachable by pinging it > 1.7.py

- The script uses the subprocess module to execute the Windows ping command from within Python

- The ping command is sent once to the target IP address

- The output of the command is captured and analyzed in background

- If the ping output contains "TTL=" this indicates a successful reply and the IP address is considered reachable

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.7.py
Enter IP address to ping: 192.168.1.1
192.168.1.1 is reachable

(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.7.py
Enter IP address to ping: 192.168.1.30
192.168.1.30 is not reachable

(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.7.py
Enter IP address to ping: 8.8.8.8
8.8.8.8 is reachable
```

8.Convert an IPv6 address to its compressed and expanded forms > 1.8.py

- Import ipaddress module.

- Take IPv6 address from user input

- Use IPv6Address() to validate and handle the address

- Print the compressed form using .compressed

- Print the expanded form using .exploded

- If the input is not a valid IPv6 address print an error message

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.8.py
Enter an IPv6 address: 2001:0db8:0000:0000:0000:ff00:0042:8329
Compressed IPv6: 2001:db8::ff00:42:8329
Expanded IPv6  : 2001:0db8:0000:0000:0000:ff00:0042:8329

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.8.py
Enter an IPv6 address: 2001:db8::ff00:42:8329
Compressed IPv6: 2001:db8::ff00:42:8329
Expanded IPv6  : 2001:0db8:0000:0000:0000:ff00:0042:8329
```

9.Write a function that determines the class of an IPv4 address (A, B, C, D, E) > 1.9.py

- The class of an IPv4 address is determined by the value of the first octet

- The script extracts the first octet of the IP address and compares it with predefined ranges to identify whether the address belongs to Class A, B, C, D, or E

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.9.py
Enter IPv4 address: 192.168.1.1
IP Class: Class C

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.9.py
Enter IPv4 address: 10.0.0.1
IP Class: Class A
```

10.Given two IPs, determine if they belong to the same subnet > 1.10.py

- The program determines whether two IP addresses belong to the same subnet

- Since no subnet mask is provided a /24 subnet is assumed

- If the first three octets of both IP addresses are the same, the IPs are considered to be in the same subnet

Screenshot:

```
(venv)[redacted]s\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.10.py
Enter first IP: 192.168.1.10
Enter second IP: 192.168.1.200
They are in the same subnet.

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 1>python3 1.10.py
Enter first IP: 192.168.1.10
Enter second IP: 192.168.2.5
They are NOT in the same subnet.
```

Day 2: Ports & Protocols

Files path : davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2

1.Print a list of common ports and their corresponding services (e.g., 80 → HTTP) > 2.1.py

- A dictionary is used to store common port numbers and their corresponding services

- The script loops through the dictionary and prints each port with its associated service

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.1.py
Port 21 → FTP
Port 22 → SSH
Port 23 → Telnet
Port 25 → SMTP
Port 53 → DNS
Port 80 → HTTP
Port 110 → POP3
Port 143 → IMAP
Port 443 → HTTPS
```

2.Write a function that checks if a given port number is valid (0-65535)

- A valid port number must be numeric and fall within the range 0 to 65535

- If the input meets these conditions the port is considered valid

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.2.py
Enter port number: 80
Valid port number

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.2.py
Enter port number: 70000
Invalid port number
```

3.Write a script that scans the first 1000 ports of a given IP to see if they are open > 2.3.py

- The script scans the first 1000 ports of a given IP address using the socket

- For each port, a connection attempt is made with a short timeout

- If the connection succeeds the port is considered open and is printed to the screen

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.3.py
Enter IP address to scan: 192.168.1.1
Scanning first 1000 ports...
Port 53 is OPEN
Port 443 is OPEN
```

4.Detects whether a given port uses TCP or UDP > 2.4.py

- The script determines whether a port commonly uses TCP, UDP, or both protocols

- This is done by comparing the given port number with a list of well-known ports and their associated transport protocols

- If the port is not found in the list it is considered unknown

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.4.py
Enter port number: 80
Protocol: TCP (HTTP)

(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.4.py
Enter port number: 161
Protocol: UDP (SNMP)

(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.4.py
Enter port number: 404
Protocol: Unknown service or protocol
```

5.Randomly generate 5 open ports between 1024-65535 > 2.5.py

- The script randomly generates five port numbers within the range 1024 to 65535

- The random library is used to ensure that the generated port numbers are unique and within the valid range

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.5.py
Random open ports:
6368
11357
14624
13947
8179
```

6.Create a function that tells whether a given port is in the privileged range (0-1023) > 2.6.py

- The function checks whether a given port number belongs to the privileged port range

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.6.py
Enter port number: 80
Privileged port

(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.6.py
Enter port number: 8080
Non-privileged port
```

7.Write a script that finds all listening ports on your local machine (127.0.0.1) > 2.7.py

- I used the external Python library psutil to detect all listening ports on the local machine

- The library was installed inside a virtual environment (venv)

- To generate listening ports for testing I started a simple Python server which opened additional ports on the local host

- Ports bound to both 127.0.0.1 and 0.0.0.0 were included to ensure that all locally listening services were captured

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.7.py
Listening ports on 127.0.0.1 and 0.0.0.0 :
135
4503
5040
49666
49665
20751
5357
8000
5354
445
49671
49667
49668
49664
7680
```

8.Implement a basic TCP port scanner using Python's socket module > 2.8.py

- This task uses the same basic TCP scanning script from task 3, since it already implements a simple TCP port scanner using Python's socket module

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.3.py
Enter IP address to scan: 192.168.1.1
Scanning first 1000 ports...
Port 53 is OPEN
Port 443 is OPEN
```

9.Implement a basic UDP port scanner > 2.9.py

- I implemented a basic UDP port scanner using Python's socket module

- The script sends a UDP packet to each port within a specified range and waits for a response

- Due to the nature of UDP the absence of a response does not always indicate that the port is closed

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.9.py
Enter IP address: 192.168.1.1
Scanning UDP ports on 192.168.1.1...
Port 1 is OPEN or FILTERED (UDP)
Port 2 is OPEN or FILTERED (UDP)
Port 3 is OPEN or FILTERED (UDP)
Port 4 is OPEN or FILTERED (UDP)
Port 5 is OPEN or FILTERED (UDP)
Port 6 is OPEN or FILTERED (UDP)
Port 7 is OPEN or FILTERED (UDP)
Port 8 is OPEN or FILTERED (UDP)
Port 9 is OPEN or FILTERED (UDP)
Port 10 is OPEN or FILTERED (UDP)
Port 11 is OPEN or FILTERED (UDP)
```

10.List all reserved ports (1-1023) that are commonly used in hacking > 2.10.py

- These ports are commonly targeted during security assessments due to the critical services they host

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 2>python3 2.10.py
Reserved ports commonly targeted:

Port 21 -> FTP
Port 22 -> SSH
Port 23 -> Telnet
Port 25 -> SMTP
Port 53 -> DNS
Port 69 -> TFTP
Port 80 -> HTTP
Port 110 -> POP3
Port 111 -> RPC
Port 123 -> NTP
Port 137 -> NetBIOS
Port 138 -> NetBIOS
Port 139 -> NetBIOS
Port 143 -> IMAP
Port 161 -> SNMP
Port 389 -> LDAP
Port 443 -> HTTPS
Port 445 -> SMB
Port 512 -> Rexec
Port 513 -> Rlogin
Port 514 -> Rsh
Port 636 -> LDAPS
Port 873 -> rsync
Port 993 -> IMAPS
Port 995 -> POP3S
```

Day 3: TCP & UDP

Files path : davinci0x01_lvl0_Tasks\Skill_2_Network\Day 3

1.Write a simple TCP client-server program that sends a message from client to server > 3.1.py

- In this task i created a simple TCP client-server program using socket module

- The server binds to 127.0.0.1 on port 5000 and waits for a connection

- The client connects to the server and sends a message

- The server receives the message and prints it

Screenshot:

2.Modify the above program to work with UDP instead of TCP > 3.2.py

- In this task i modified the previous TCP client-server program to work using UDP

- Unlike TCP, UDP does not require a connection so the server does not use listen() or accept()

- The client sends a message using sendto() and the server receives it using recvfrom()

Screenshot:

3.Create a TCP server that listens on port 9999 and responds with "Hello, Client!" > 3.3.py

- In this task i created a TCP server that listens on port 9999

- When a client connects the server sends the message "Hello, Client!" and then closes the connection

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_
2_Network\Day 3>python3 3.3.py
Server listening on port 9999...
Connected by: ('127.0.0.1', 50804)
```

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_
2_Network>python3 3.3-client.py
Server says: Hello, Client!
```

4.Write a function that measures the round-trip time (RTT) of a TCP connection > 3.4.py

- The RTT represents the time it takes for a connection request to be sent to a remote server and for the response to be received

- To calculate this the script records the current time before attempting to establish a TCP connection Once the connection is successfully established the time is recorded again and the difference between the two timestamps represents the RTT

- The result is converted to milliseconds to provide a clearer measurement of network delay

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 3>python3 3.4.py
RTT to 8.8.8.8:53 = 56.38 ms
```

5.Simulate a basic SYN flood attack (for controlled environments) > 3.5.py

- The simulation demonstrates a basic SYN flood attack by sending a large number of TCP SYN packets with spoofed source IP addresses to a target server

- I used Scapy library to craft and send custom TCP packets where only the SYN flag is set without completing the three-way handshake

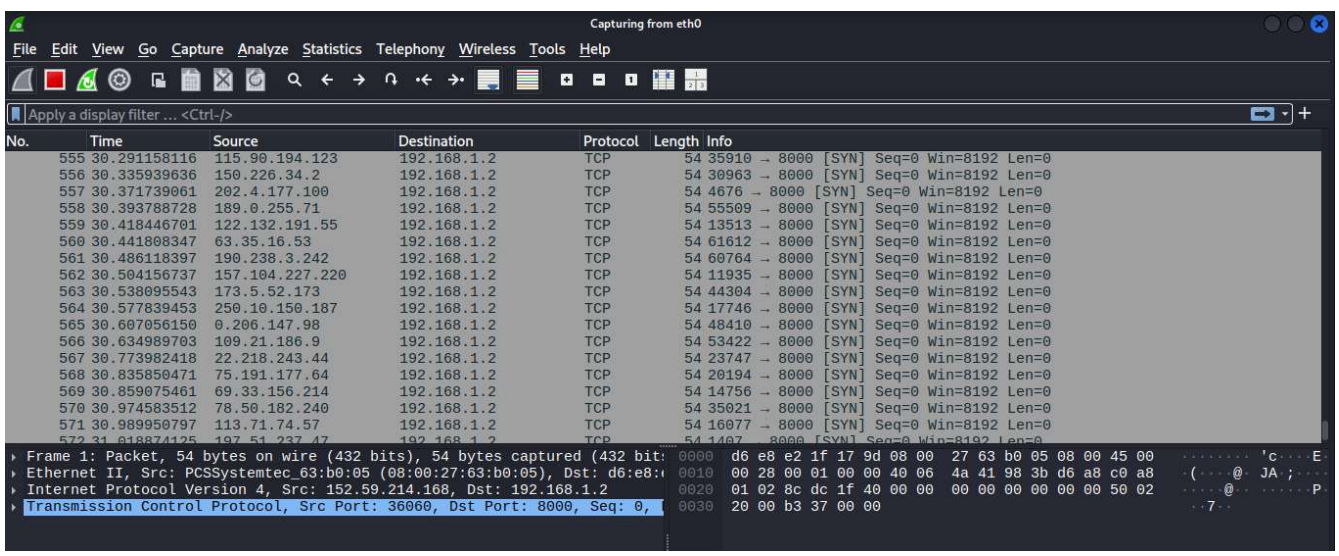
- Random source IP addresses are generated for each packet to mimic distributed traffic and make tracing difficult

- The target machine runs a simple Python HTTP server listening on port 8000 which accepts incoming connection requests

- During the attack, Wireshark captures incoming SYN packets from various spoofed source IPs all targeting the same destination IP and port

- No ACK packets are sent back from the attacker leaving the target with many half-open connections waiting for responses that never arrive

Screenshot:



6.Write a script that monitors and logs all incoming TCP connections > 3.6.py

- The server listens on port 8000 and records the client IP address, source port, and timestamp

- In this task, I created a TCP monitoring script using Python's socket module

- The server listens on port 8080 and logs every incoming connection with the client IP and timestamp

- For testing, I used the Telnet command to connect to the server (telnet <server_ip> 8080)

- Each Telnet connection created a new incoming TCP connection and the script successfully recorded it in the console and in a log file > connections.log

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 3>python3 3.6.py
Monitoring incoming TCP connections on port 8080...
[2026-02-11 17:20:15.1] Connection from 192.168.1.2
[2026-02-11 17:20:22.1] Connection from 192.168.1.2
[2026-02-11 17:20:23.2] Connection from 192.168.1.2
[2026-02-11 17:20:24.1] Connection from 192.168.1.2
[2026-02-11 17:20:25.0] Connection from 192.168.1.2
```

7.Implement a TCP handshake simulation in Python > 3.7.py

- In this task i simulated the TCP 3-way handshake in Python using a client and server

- The client sends a "SYN" message and the server responds with "SYN-ACK" and then the client sends "ACK"

- The real TCP handshake happens automatically when the client connects and the server accepts the connection but I added a small message exchange to represent the handshake steps clearly

- The SO_REUSEADDR option was enabled to allow the server to reuse the same port immediately after restarting, preventing "address already in use" errors

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 3>python3 3.7.py
Client: sent SYN
Server: received SYN
Server: sent SYN-ACK
Client: received SYN-ACK
Client: sent ACK
Client: Handshake completed ✅
Server: received ACK
Server: Handshake completed ✅
```

8.Create a script that tests whether a remote server allows telnet access (port 23) > 3.8.py

- In this task i wrote a Python script to check whether a remote server allows Telnet access on port 23

- The script attempts to establish a TCP connection to the target IP on port 23

- If the connection succeeds the port is considered open otherwise, it is closed

Screenshot:

```
(venv) [redacted]s\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 3>python3 3.8.py
Enter target IP: 192.168.1.1
Telnet (port 23) is CLOSED on 192.168.1.1
```

9.. Write a UDP client that sends a DNS request manually to 8.8.8.8 > 3.9.py

- In this task i created a UDP client in Python that sends a DNS request manually to Google Public DNS (8.8.8.8) on port 53

- Instead of using a DNS library i built the DNS packet manually as raw bytes to understand the DNS message format

- Header (12 bytes): Contains fields like Transaction ID, Flags, and counters (QDCOUNT, ANCOUNT, NSCOUNT, ARCOUNT)

- Question Section: Contains the domain name (QNAME) ,query type (QTYPE) and query class (QCLASS)

- A random Transaction ID (2 bytes) was generated to match the request with its response

- The Flags field was set to 0x0100 to represent a standard query with recursion desired

- QDCOUNT = 1 (one question) and ANCOUNT, NSCOUNT, ARCOUNT = 0 (no answers in the request)

- The domain name was encoded in DNS format by splitting it into labels and prefixing each label with its length Example: example.com becomes \x07example\x03com\x00

- QTYPE was set to A (1) to request an IPv4 address and QCLASS was set to IN (1) for Internet

- A UDP socket was created and the request was sent using sendto() to (8.8.8.8, 53)

- The script then waited for a response using recvfrom() with a timeout to avoid hanging.

- If a response was received the script printed the number of bytes received confirming that the DNS server processed the request

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_2_Network\Day 3>python3 3.9.py
Received 61 bytes from DNS server.
```

10.Implement a simple packet sniffer that captures TCP and UDP packets > 3.10.py

Note: I used Kali Linux to run this script

- The simple packet sniffer captures and displays TCP and UDP packets passing through the network interface

- Scapy library is used to extract IP, TCP and UDP layers from each captured packet

- For TCP packets it shows source IP:port -> destination IP:port and the TCP flags (S, A, PA, etc.)

- For UDP packets it shows source IP:port -> destination IP:port only

Screenshot:

```
┌──(kali㉿kali)-[~/Downloads]
└─$ sudo python3 sniffer.py
Simple TCP/UDP Sniffer Started
Press Ctrl+C to stop
────────────────────────────────────────────────────────────
[UDP] 192.168.1.5:53499 → 104.18.28.234:443
[UDP] 192.168.1.5:53499 → 104.18.28.234:443
[UDP] 192.168.1.5:53499 → 104.18.28.234:443
[UDP] 104.18.28.234:443 → 192.168.1.5:53499
[UDP] 104.18.28.234:443 → 192.168.1.5:53499
[UDP] 104.18.28.234:443 → 192.168.1.5:53499
[UDP] 104.18.28.234:443 → 192.168.1.5:53499
[UDP] 192.168.1.5:53499 → 104.18.28.234:443
[TCP] 172.64.148.235:443 → 192.168.1.5:60982   Flags: PA
[TCP] 192.168.1.5:60982 → 172.64.148.235:443   Flags: PA
[TCP] 172.64.148.235:443 → 192.168.1.5:60982   Flags: A
^C
```